

PES UNIVERSITY
Department of Computer Science and Engineering

SOFTWARE ENGINEERING LAB – LAB 07

FUZZ TESTING FOR INPUT VALIDATION BUGS

Application: Phone Repair Quote | Tools: Python, Pytest, Hypothesis

Name : Chaitanya Krishna M

SRN : PES1UG24CS122

Semester : 5th Sem

Section : B

LAB 7 | FUZZ TESTING

Phone Repair Quote – Hypothesis property-based fuzzing of input validation

1. Objective

Fuzz-test the Lab 6 Phone Repair Quote application (`processor.py`) with Hypothesis, identify input validation bugs in its three functions, fix them, and show the fixed code passing all tests.

2. Files

File	Purpose
processor.py	Application with input-validation bugs fixed
original/processor.py	Original Lab 6 code (used for the failing run)
test_processor.py	Hypothesis fuzz tests for repair_quote, repair_days, trade_in_value (10 tests)

3. Bugs found by fuzzing

Function	Failing input	Original behaviour	Fix
repair_quote	part=[] / {}	TypeError: unhashable type	Check part is str before lookup
repair_quote	non-bool warranty / express	Silently treated as truthy	Require bool, else ValueError
repair_days	technicians_free=-1	Accepted, returns a day count	Require int >= 0
repair_days	unknown part, non-bool in-stock	Accepted	Validate part and flag
trade_in_value	None / text price or age	TypeError from comparison	Require int, else ValueError
trade_in_value	age=0.0, nan, inf, 1e308	Accepted / returns nan	Whole-number price and age only

4. Final results

Run	Result
Original code	6 failed, 4 passed
Corrected code	10 passed

5. Repository

Repository	Link
Forked repository	https://github.com/Chaitanya-171005/L6_L7_59_phone_repair_quote (folder LAB7)

RESULT 1 | Original Application

Execution of the original application (original/processor.py)

Terminal output

```
(base) apple@Chaitanyas-MacBook-Pro original % cat >> processor.py <<'EOF'


if __name__ == "__main__":
    print(repair_quote("screen", "budget", False, False))
    print(repair_days("screen", False, 0))
    print(trade_in_value(50000, 10, False))
EOF
(base) apple@Chaitanyas-MacBook-Pro original % python processor.py
4600
8
30000
(base) apple@Chaitanyas-MacBook-Pro original %
```

Original application run: repair_quote = 4600, repair_days = 8, trade_in_value = 30000.

RESULT 2 | Failing Test Cases

Hypothesis fuzz tests run against the original code

Terminal output

```
(base) apple@Chaitanyas-MacBook-Pro LAB7 % pytest -v --tb=line
============================= test session starts ==============================
platform darwin -- Python 3.13.9, pytest-9.1.1, pluggy-1.7.0 -- /opt/miniconda3/bin/python3.13
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7
plugins: hypothesis-6.170.15, cov-7.1.0, django-4.14.0, anyio-4.15.1
collected 10 items

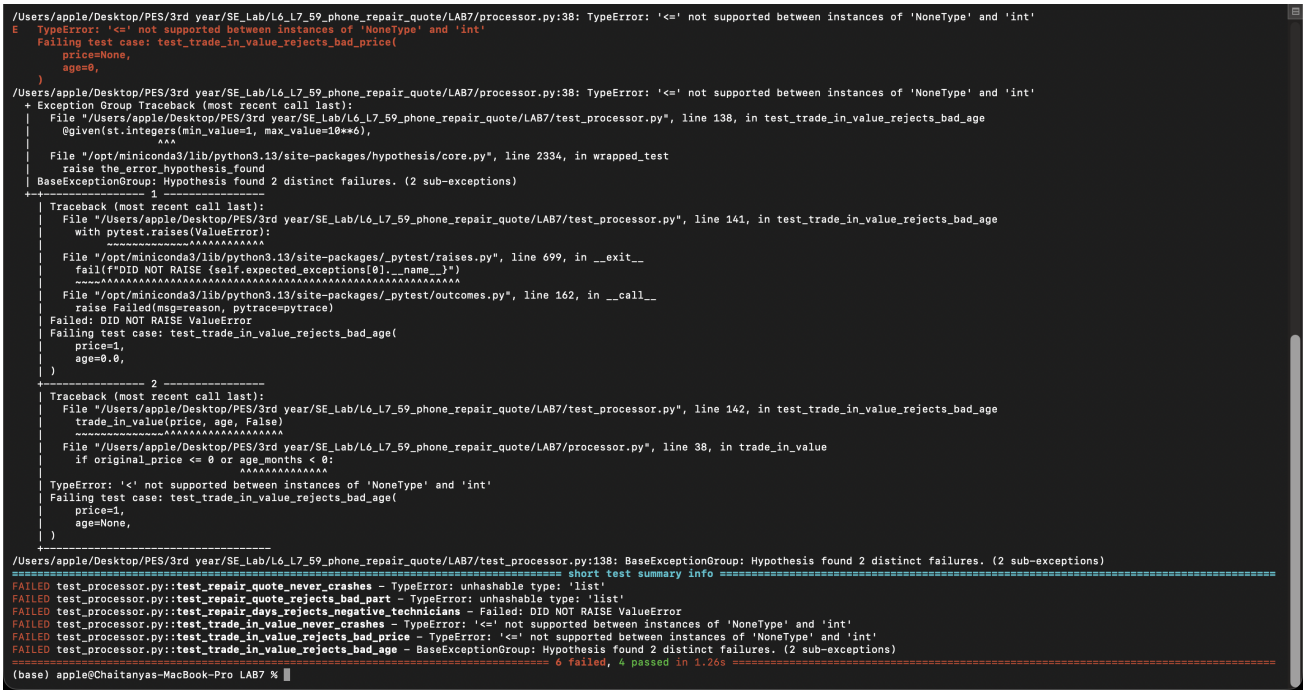
test_processor.py::test_repair_quote_never_crashes FAILED                [ 10%]
test_processor.py::test_repair_quote_valid_inputs PASSED                 [ 20%]
test_processor.py::test_repair_quote_rejects_bad_part FAILED             [ 30%]
test_processor.py::test_repair_days_never_crashes PASSED                 [ 40%]
test_processor.py::test_repair_days_valid_inputs PASSED                  [ 50%]
test_processor.py::test_repair_days_rejects_negative_technicians FAILED  [ 60%]
test_processor.py::test_trade_in_value_never_crashes FAILED              [ 70%]
test_processor.py::test_trade_in_value_valid_inputs PASSED               [ 80%]
test_processor.py::test_trade_in_value_rejects_bad_price FAILED          [ 90%]
test_processor.py::test_trade_in_value_rejects_bad_age FAILED            [100%]

=================================== FAILURES ===================================
E   TypeError: unhashable type: 'list'
    Failing test case: test_repair_quote_never_crashes(
        part=[],
        tier=None,
        warranty=None,
        express=False,
    )
/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/processor.py:5: TypeError: unhashable type: 'list'
E   TypeError: unhashable type: 'list'
    Failing test case: test_repair_quote_rejects_bad_part(
        part=[],
    )
/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/processor.py:5: TypeError: unhashable type: 'list'
E   Failed: DID NOT RAISE ValueError
    Failing test case: test_repair_days_rejects_negative_technicians(
        part='screen',
        in_stock=False,
        techs=-1,
    )
/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/test_processor.py:100: Failed: DID NOT RAISE ValueError
E   TypeError: '<=' not supported between instances of 'NoneType' and 'int'
    Failing test case: test_trade_in_value_never_crashes(
        price=None,
        age=None,
        cracked=None,
    )
/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/processor.py:38: TypeError: '<=' not supported between instances of 'NoneType' and 'int'
E   TypeError: '<=' not supported between instances of 'NoneType' and 'int'
    Failing test case: test_trade_in_value_rejects_bad_price(
        price=None,
        age=0,
    )
```

pytest -v --tb=line on the original code: test list and first failures.

RESULT 2 | Failing Test Cases (continued)

Remaining failures and summary


/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/processor.py:38: TypeError: '<=' not supported between instances of 'NoneType' and 'int'
E TypeError: '<=' not supported between instances of 'NoneType' and 'int'
Failing test case: test_trade_in_value_rejects_bad_price(
price=None,
age=0,
)
/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/processor.py:38: TypeError: '<=' not supported between instances of 'NoneType' and 'int'
+ Exception Group Traceback (most recent call last):
| File "/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/test_processor.py", line 138, in test_trade_in_value_rejects_bad_age
| @given(st.integers(min_value=1, max_value=10**6),
| ^^^
| File "/opt/miniconda3/lib/python3.13/site-packages/hypothesis/core.py", line 2334, in wrapped_test
| raise the_error_hypothesis_found
| BaseExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
+-+---------- 1 ----------
| Traceback (most recent call last):
| File "/Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7/test_processor.py", line 141, in test_trade_in_value_rejects_bad_age
| with pytest.raises(ValueError):
| ^^^

RESULT 3 | Corrected Application

Execution of the corrected application (processor.py)

Terminal output

```
(base) apple@Chaitanyas-MacBook-Pro LAB7 % python processor.py
4600
8
30000
(base) apple@Chaitanyas-MacBook-Pro LAB7 %
```

Corrected application run: same valid-input results (4600, 8, 30000).

RESULT 4 | Final Test Execution

All Hypothesis fuzz tests pass on the corrected code

```
(base) apple@Chaitanyas-MacBook-Pro LAB7 % pytest -v
============================= test session starts ==============================
platform darwin -- Python 3.13.9, pytest-9.1.1, pluggy-1.7.0 -- /opt/miniconda3/bin/python3.13
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/apple/Desktop/PES/3rd year/SE_Lab/L6_L7_59_phone_repair_quote/LAB7
plugins: hypothesis-6.170.15, cov-7.1.0, django-4.14.0, anyio-4.15.1
collected 10 items

test_processor.py::test_repair_quote_never_crashes PASSED                [ 10%]
test_processor.py::test_repair_quote_valid_inputs PASSED                 [ 20%]
test_processor.py::test_repair_quote_rejects_bad_part PASSED             [ 30%]
test_processor.py::test_repair_days_never_crashes PASSED                 [ 40%]
test_processor.py::test_repair_days_valid_inputs PASSED                  [ 50%]
test_processor.py::test_repair_days_rejects_negative_technicians PASSED  [ 60%]
test_processor.py::test_trade_in_value_never_crashes PASSED              [ 70%]
test_processor.py::test_trade_in_value_valid_inputs PASSED               [ 80%]
test_processor.py::test_trade_in_value_rejects_bad_price PASSED          [ 90%]
test_processor.py::test_trade_in_value_rejects_bad_age PASSED            [100%]

============================== 10 passed in 3.32s ==============================
(base) apple@Chaitanyas-MacBook-Pro LAB7 %
```

pytest -v on the corrected code: 10 passed.

SOURCE CODE | processor.py

Application with input validation fixes

```
"""Phone repair quote module (Lab 6 application, fixed for Lab 7 fuzz-found bugs).

Every function validates its input and raises ValueError for anything
outside its documented domain, instead of crashing with TypeError or
silently returning garbage (negative technicians, NaN/inf prices, ...).
"""

PART_COST = {"screen": 4000, "battery": 1500, "camera": 2500, "charging_port": 800}


def _is_int(value):
    """True for real integers; bool is a subclass of int, so exclude it."""
    return isinstance(value, int) and not isinstance(value, bool)


def _check_part(part):
    # isinstance check first: an unhashable value (list/dict) would make
    # `part in PART_COST` raise TypeError instead of ValueError.
    if not isinstance(part, str) or part not in PART_COST:
        raise ValueError("unknown part")


def _check_bool(value, name):
    if not isinstance(value, bool):
        raise ValueError("invalid " + name)


def repair_quote(part, brand_tier, under_warranty, express):
    _check_part(part)
    if not isinstance(brand_tier, str) or brand_tier not in ("budget", "flagship"):
        raise ValueError("unknown brand tier")
    _check_bool(under_warranty, "under_warranty")
    _check_bool(express, "express")
    cost = PART_COST[part]
    if brand_tier == "flagship":
        cost = cost * 2
    if part == "screen":
        labour = 600
    else:
        labour = 300
    if under_warranty and part != "screen":
        cost = 0
    total = cost + labour
    if express:
        total += 500
    return total


def repair_days(part, parts_in_stock, technicians_free):
    _check_part(part)
    _check_bool(parts_in_stock, "parts_in_stock")
    if not _is_int(technicians_free) or technicians_free < 0:
        raise ValueError("invalid technicians_free")
    if technicians_free == 0:
        days = 3
    else:
        days = 1
    if not parts_in_stock:
        if part == "screen" or part == "camera":
            days += 5
        else:
            days += 2
    return days


def trade_in_value(original_price, age_months, screen_cracked):
    # Whole-number price/age only: floats let NaN, inf and overflow
    # (1e308 * 60 -> inf -> nan) slip through the old `<= 0` checks.
    if not _is_int(original_price) or not _is_int(age_months):
        raise ValueError("invalid input")
    _check_bool(screen_cracked, "screen_cracked")
    if original_price <= 0 or age_months < 0:
        raise ValueError("invalid input")
    if age_months > 48:
        return 0
    if age_months <= 12:
        value = original_price * 60 // 100
    elif age_months <= 24:
        value = original_price * 40 // 100
    else:
        value = original_price * 20 // 100
    if screen_cracked:
        value = value // 2
    return value


if __name__ == "__main__":
    print(repair_quote("screen", "budget", False, False))
    print(repair_days("screen", False, 0))
    print(trade_in_value(50000, 10, False))
```

SOURCE CODE | test_processor.py

Hypothesis fuzz tests for all three functions

```
"""Lab 7 - Hypothesis fuzz tests for processor.py (all three functions).

Two kinds of property per function:
  1. "Never crashes": for ANY input, the function either returns a value or
     raises ValueError -- never TypeError, never an unhandled exception.
  2. "Correct on valid input": for inputs inside the valid domain, the result
     satisfies the business rules.
"""
import pytest
from hypothesis import given, settings, strategies as st

from processor import PART_COST, repair_quote, repair_days, trade_in_value

PARTS = list(PART_COST)
TIERS = ["budget", "flagship"]

# Anything at all: None, bools, ints, floats (nan/inf), text, bytes, lists, dicts...
anything = st.one_of(
    st.none(),
    st.booleans(),
    st.integers(),
    st.floats(allow_nan=True, allow_infinity=True),
    st.text(),
    st.binary(),
    st.lists(st.integers()),
    st.dictionaries(st.text(), st.integers()),
    st.sampled_from(PARTS + TIERS),
)

# Mostly *valid* values mixed with junk, to reach deeper code paths.
partish = st.one_of(st.sampled_from(PARTS), anything)
tierish = st.one_of(st.sampled_from(TIERS), anything)
boolish = st.one_of(st.booleans(), anything)

FUZZ = settings(max_examples=500, deadline=None)


def only_valueerror(fn, *args):
    """Call fn; only a result or ValueError is acceptable."""
    try:
        return fn(*args)
    except ValueError:
        return None


# ---------------------------------------------------------------- repair_quote
@FUZZ
@given(partish, tierish, boolish, boolish)
def test_repair_quote_never_crashes(part, tier, warranty, express):
    result = only_valueerror(repair_quote, part, tier, warranty, express)
    if result is not None:
        assert isinstance(result, int) and not isinstance(result, bool)
        assert result >= 300


@FUZZ
@given(st.sampled_from(PARTS), st.sampled_from(TIERS), st.booleans(), st.booleans())
def test_repair_quote_valid_inputs(part, tier, warranty, express):
    total = repair_quote(part, tier, warranty, express)
    labour = 600 if part == "screen" else 300
    assert total >= labour
    assert total <= PART_COST[part] * 2 + labour + 500
    # express always adds exactly 500
    assert total - repair_quote(part, tier, warranty, False) == (500 if express else 0)
    # flagship never cheaper than budget
    assert repair_quote(part, "flagship", warranty, express) >= repair_quote(part, "budget", warranty, express)


@FUZZ
@given(st.one_of(st.lists(st.integers()), st.dictionaries(st.text(), st.integers()),
                 st.none(), st.integers(), st.text().filter(lambda s: s not in PART_COST)))
def test_repair_quote_rejects_bad_part(part):
    with pytest.raises(ValueError):
        repair_quote(part, "budget", False, False)


# ---------------------------------------------------------------- repair_days
@FUZZ
@given(partish, boolish, anything)
def test_repair_days_never_crashes(part, in_stock, techs):
    result = only_valueerror(repair_days, part, in_stock, techs)
    if result is not None:
        assert 1 <= result <= 8


@FUZZ
@given(st.sampled_from(PARTS), st.booleans(), st.integers(min_value=0, max_value=10**6))
def test_repair_days_valid_inputs(part, in_stock, techs):
    days = repair_days(part, in_stock, techs)
    assert 1 <= days <= 8
```

```
    # no free technician is never faster than having one
    assert repair_days(part, in_stock, 0) >= repair_days(part, in_stock, 1)
    # parts out of stock is never faster than in stock
    assert repair_days(part, False, techs) >= repair_days(part, True, techs)


@FUZZ
@given(st.sampled_from(PARTS), st.booleans(), st.integers(max_value=-1))
def test_repair_days_rejects_negative_technicians(part, in_stock, techs):
    with pytest.raises(ValueError):
        repair_days(part, in_stock, techs)


# ------------------------------------------------------------ trade_in_value
@FUZZ
@given(anything, anything, boolish)
def test_trade_in_value_never_crashes(price, age, cracked):
    result = only_valueerror(trade_in_value, price, age, cracked)
    if result is not None:
        # never nan/negative, and never worth more than the original price
        assert isinstance(result, int)
        assert 0 <= result <= price


@FUZZ
@given(st.integers(min_value=1, max_value=10**9), st.integers(min_value=0, max_value=200), st.booleans())
def test_trade_in_value_valid_inputs(price, age, cracked):
    value = trade_in_value(price, age, cracked)
    assert isinstance(value, int)
    assert 0 <= value <= price * 60 // 100
    if age > 48:
        assert value == 0
    # a cracked screen never increases the value
    assert trade_in_value(price, age, True) <= trade_in_value(price, age, False)
    # older phones are never worth more
    assert trade_in_value(price, age + 1, cracked) <= value


@FUZZ
@given(st.one_of(st.floats(), st.integers(max_value=0), st.none(), st.text()),
       st.integers(min_value=0, max_value=100))
def test_trade_in_value_rejects_bad_price(price, age):
    with pytest.raises(ValueError):
        trade_in_value(price, age, False)


@FUZZ
@given(st.integers(min_value=1, max_value=10**6),
       st.one_of(st.integers(max_value=-1), st.floats(), st.none(), st.text()))
def test_trade_in_value_rejects_bad_age(price, age):
    with pytest.raises(ValueError):
        trade_in_value(price, age, False)
```